Лаба по сортам

Цель данной лабораторной работы – посмотреть на алгоритмы с различной асимптотикой, научиться анализировать время работы алгоритмов и включать разные степени оптимизации. Заодно, это – практика написания кода по описанному алгоритму. Рекомендуем писать код самостоятельно. Если напишете быстро, то много времени не потратите, а если закопаетесь – ну, значит, практика вам явно пойдет на пользу.

По возможности, постарайтесь сделать свою жизнь легче заранее. Планируйте код таким образом, чтобы с ним было удобно работать – меньше переписывать между пунктами или при замене размера массива. Используйте свои наработки и готовые куски кода. Можете заранее прочитать весь текст лабы, чтобы спланировать работу над кодом, это тоже лишним не будет.

Можете использовать стартер-коды, которые идут дальше. Это небольшие работающие кусочки кода, которые реализуют функционал, который может вам пригодиться. Можете пока использовать их как магические формулы, не пытаясь разбираться в деталях, как они устроены. Полностью мы их разберем к середине следующего семестра.

Стартер-коды

Стартер-код для вывода в файл:

```cpp
#include <fstream>
using namespace std;
int main()
{
    ofstream f("1.csv", ios::out);
    // csv - стандартный формат для хранения данных прямым текстом.
    f << "forty two is six times seven" << endl; // работаете как с привычным cout
    return 0;
}
```

Стартер-код для измерения времени:

```cpp
#include <iostream>
#include <chrono>
int main() {
    auto start = std::chrono::high_resolution_clock::now(); // записали первый таймкод
    for (long i = 1; i < 1000000000; i += 1); // здесь то что вы хотите измерить
    auto end = std::chrono::high_resolution_clock::now(); // записали второй таймкод
    auto nsec = end - start; // нашли разницу между таймкодами
    std::cout << nsec.count() << " нсек." << std::endl;
    return 0;
}
```

Функция для измерения времени (возвращает время в секундах, можно использовать ее вместо первого примера):

```cpp
#include <chrono>
double get_time() {
    return std::chrono::duration_cast<std::chrono::microseconds>
        (std::chrono::steady_clock::now().time_since_epoch()).count()/1e6;
}
```

Функция для произвольных чисел (возвращает произвольное целое число в диапазоне от min до max, ее тоже можно использовать, не влезая в ее внутренний мир):

```cpp
#include <chrono>
#include <random>
int rand_uns(int min, int max) {
    unsigned seed = std::chrono::steady_clock::now().time_since_epoch().count();
    static std::default_random_engine e(seed);
    std::uniform_int_distribution<int> d(min, max);
    return d(e);
}
```

Графики стройте с помощью python (matplotlib). Напоминаем про примеры кода с буткемпа:

https://github.com/TheEntityCircle/boopcamp-python

Первая арка (попроще)

0. Простые медленные сортировки

Начнем с простого. Напишите три разных сортировки с временной сложностью O(N^2). Это может быть пузырек, вставка, шейкер, простой выбор, непростой выбор или что угодно, что вы еще найдете. Постройте график зависимости времени работы от размера массива с тремя кривыми – по одной для каждой сортировки. Эти три кривые лучше нарисовать на одном графике, чтобы было удобнее сравнивать, какой алгоритм быстрее.

Постройте те же самые зависимости в двойном логарифмическом масштабе и проверьте таким образом, что сложность этих алгоритмов действительно O(N^2).

Что нужно показать:

- график с тремя кривыми в обычном масштабе;
- график с тремя кривыми в двойном логарифмическом масштабе.

1. Автоматическая оптимизация

Каждый раз, когда вы пишете какой-то код и хотите его запустить, на самом деле, это происходит в два этапа. Вначале компилятор делает сборку и получает исполняемый файл (в Windows у него будет расширение .exe). Потом происходит запуск исполняемого файла. Поэтому в любой уважающей себя IDE это две разные кнопки - build и run. Если код вы не меняли и просто хотите несколько раз запустить с разным вводом, то перекомпилировать его не надо, только перезапустить. Пока что разница небольшая, а в будущем семестре вы заметите, что ваш код целиком может и несколько минут перекомпилироваться (а кто-то, может, и до часов дойдет).

Более детально в процесс сборки мы заглянем в будущем семестре, а пока что нам важно, что на этом этапе компилятор разбирает ваш код и пытается сложить из него подробные инструкции для процессора (какие числа откуда брать и какие арифметические операции над ними проводить). Если вы ничего особо не трогали и просто нажали на кнопку, то инструкции получатся не очень эффективными. Где-то числа будут иногда гоняться между ячейками туда-сюда, где-то использоваться медленные инструкции вместо быстрых.

При этом внутри компилятора есть большой набор алгоритмов, каждый из которых может сделать ваш код быстрее. В детали можно не погружаться (большая часть программистов на C++ не погружается) и включать их крупными блоками со стандартными названиями.

Если вы компилируете g++ и запускаете получившийся бинарник из консоли, то добавляйте при компиляции флаги -O0, -O1, -O2 и -O3 соответственно. Если работаете в среде разработки (Code::Blocks, CLion, Microsoft Visual Studio/Code, ...), то вам нужно найти галочки с такими же названиями в настройках компилятора.

Если у вас CMake (CLion/MVS Code), то нужно дописать в CMakeLists.txt эту строку:

```
set(CMAKE_CXX_FLAGS "-O2")
```

Если у вас Code::Blocks, то там есть коварство в интерфейсе. Верхние тычки не работают: если вы их ткнете, то ничего не изменится. Вам нужны те, которые ниже и подписаны подробно.

Если у вас Microsoft Visual Studio (не Code), то флаги будут называться немного иначе. Ориентируйтесь на описание.

Постройте график с пятью кривыми для одной из сортировок за O(N^2) – по одной кривой на каждую степень оптимизации и одна без флагов.

Если вы ничего такого не делали, а у вас пошла стопка нулей вместо измерений времени, могло произойти следующее. Оптимизация O3 вырезает куски кода, если они вычисляют значения, которые потом не выходят за пределы программы. Ну, например, всю вашу сортировку – логично, если на экран вы выводите только время, то само содержимое массива и процесс его обработки компилятор считает себя вправе вырезать. Обходить это можно разными способами – можно выводить что-то из массива на экран, можно отмечать ваш контейнер как volatile (ключевое слово перед объявлением переменной). Поэкспериментируйте. Игры в армрестлинг с O3 помогают неплохо понять некоторые ее аспекты. Это злая штука. В некоторых редких случаях она может сломать код (очень редко, очень сложно такое поймать – обычно это или при ассемблерных вставках, или при

адском жонглировании r-value) или испортить точность вычислений. Нормальной безопасной опцией считается O2: она обычно оптимизирует примерно так же, но гарантированно не ломает ничего.

Что нужно показать:

- график с пятью кривыми.

2. Сложные быстрые сортировки

Напишите три разных сортировки с временной сложностью O(N log N). Это может быть куча, Хоар, расческа, слияние, радикс или что угодно, что вы еще найдете. Постройте график зависимости времени работы от размера массива с тремя кривыми – по одной для каждой сортировки.

Покажите, что сложность этих алгоритмов действительно O(N log N). Для того, чтобы это доказать, проще всего свести график к какому-нибудь линейному виду. Например, построить зависимость t/NlogN от N. Как вы догадываетесь, должна получиться константа. Если придумаете что-то более элегантное, как двойной логарифмический масштаб для полинома – пишите и рассказывайте.

Что нужно показать:

- график с тремя кривыми.

3. Халк против Локи

Весьма напрашивающийся пункт. Нарисуйте кривые из пунктов 0 и 2 на одном графике. Возможно, вам для этого придется какие-то из них пересчитать, чтобы масштабы по горизонтальной оси соответствовали.

А теперь то же самое, но для остальных оптимизаций. Определите среднее значение ускорения для каждого из алгоритмов и каждой из оптимизаций.

Что нужно показать:

- четыре графика, каждый с шестью кривыми;
- табличка шесть на четыре со значениями ускорения.

4. C++ против Питона

Еще один поединок с ожидаемым исходом. Но одно дело слышать, что Питон медленнее, другое – убедиться своими глазами и посмотреть, а точно ли отличие так велико.

Если вы умеете в Питон, то напишите три простые сортировки на Питоне. Если не умеете, то полностью законно найти реализации в интернете или сгенерить нейронкой.

Измерьте время работы сортировок в зависимости от размера массива и сравните с такими же сортировками, написанными на C++.

Что нужно показать:

- три графика, показывающие отличие Питона от C++.

5. Зависимость от начальных данных

Сравните время работы алгоритмов на отсортированном массиве, массиве произвольных чисел и массиве, отсортированном в обратную сторону. То есть лучший, средний и худший случай.

Для каждого из шести реализованных алгоритмов постройте по три кривых – лучший, средний и худший случай. Подумайте, как их сгруппировать, чтобы наиболее наглядно продемонстрировать разницу – в этом пункте может получиться несколько картинок.

Только не запутайтесь во флагах и галочках, оптимизация должна быть одинаковой - вы уже догадываетесь, какой.

Что нужно показать:

- набор графиков, демонстрирующий разницу между алгоритмами.

Вторая арка (посложнее)

В этой арке нет прописанных пунктов, которые надо показать. Вам надо самим подумать над тем, как представить полученные данные.

6. Королевская битва

Чтобы двинуться дальше, надо для начала разобраться с теми кусками кода, которые вы до этого выносили за пределы измерения времени.

Измерьте время работы в зависимости от размера массива для следующих блоков кода:

- генерация массива произвольных чисел;
- арифметическая обработка всех элементов массива (любая операция) с начала до конца;
- арифметическая обработка всех элементов массива (любая операция) в обратном порядке, с конца до начала;
- вывод массива на экран.

С какой неприятностью вы можете столкнуться. Время работы какого-то куска кода может быть меньше чувствительности таймера. Тогда график будет или сливаться с нулем, или быть очень ступенчатым, перепрыгивая между 2-3-4 значениями. Не верьте chrono, что его nanoseconds это честные наносекунды с точностью до наносекунды. Он врет, он вполне может считать в десятках тысяч или миллионах наносекунд. Таймер - штука системная, и на разных машинах/ОС его точность может отличаться. В этом случае повторяйте кусок кода много раз и усредняйте по количеству повторений – 1000, 10000, 1000000 – сколько понадобится, чтобы графики стали гладкими. Да, это одна из причин автоматизировать процесс отрисовки графиков. Если до сих пор не стало больно перерисовывать стопки графиков вручную, то теперь точно станет.

7. Халк против Локи, но поменьше

А теперь подкопаемся под определение асимптотики алгоритма. Повторите то, что вы делали в пункте 3, для одного из ускорений (например, пацанский оптимум O2) но на очень маленьких размерах массива, от 3 до 100 элементов. Проанализируйте, как изменился вид графика.

А теперь добавьте туда сравнение с кодом на Питоне.

С какой неприятностью вы можете столкнуться. Не забывайте, что время работы алгоритма зависит от входных данных – если вы раз за разом сортируете уже отсортированный массив, то это будет не совсем то время, которое нам нужно. Или совсем не то, тут уж какой алгоритм. Если вы повторяете сортировку – подумайте, как каждый раз сортировать новый произвольный массив. Вы уже увидели, что рандом – штука медленная. Если вы вызываете его в рамках таймера, он перекроет все другие эффекты, которые вы пытались посмотреть. Выносите его за пределы таймера и не стесняйтесь жрать дополнительную память, в этом пункте у нас мелкие массивы.

8. Королевская битва, но поскучнее

Скорее всего, в предыдущих пунктах вы сортировали инты. А попробуйте отсортировать массивы других типов данных и сравнить время работы.

Будет наиболее наглядно, если на одном графике изобразить кривые для времени сортировки разных типов – char, short int, int, long int, long long int, а также нескольких структур. Попробуйте добавить «тяжелые» структуры – с большим количеством полей и статическими массивами.

Думаю, вы вполне догадываетесь, что маленькие элементы должны сортироваться быстрее больших, но есть нюансы. Попробуйте их изучить и подумать, откуда они берутся.